

Conception de circuits numériques complexes sur FPGA (SN422)

Cahier de Travaux Pratiques (TP)

Le cours de « Conception de circuits numériques complexes sur FPGA » (SN422) est la suite du cours : « Introduction à la conception des circuits numériques » (SN360). Il a pour but de compléter ce qui vous a été enseigné en SN360 (représentation des nombres, circuits combinatoires, circuits séquentiels, machine à états et langages de description de matériel), en introduisant :

- Des règles de conception en lien avec les défauts des circuits numériques, comme les temps de propagation, les aléas et les métastabilités.
- Des méthodes de conception avancées de circuits numériques complexes, comme la méthodologie *Algorithm State Machine with Datapath* (ASMD) et la synthèse de haut niveau (HLS : *High-Level Synthesis*).
- Des concepts liés aux circuits numériques complexes comme le parallélisme ou le *pipelining*.
- Des architectures spécifiques pour l'implémentation de filtres numériques.

Pour cela, vous avez 4 séances de Travaux Pratiques (TP) de 3h30, soit 14h au total, pour mettre en pratique ce qui vous a été enseigné dans le cadre de ce cours.

Présentation du sujet

Les TP du cours portent sur l'implémentation d'un algorithme du Plus Grand Diviseur Commun (PGCD) sous la forme d'un circuit numérique, déployé sur une carte de développement Basys 3 (FPGA : *Field Programmable Gate Area*). Pour cela, vous allez utiliser et comparer deux méthodes de conception avancées pour implémenter l'algorithme du PGCD sous la forme d'un circuit numérique complexe. D'une part, vous allez implémenter ce circuit à l'aide de la méthode ASMD. D'autre part, vous allez comparer votre circuit avec celui obtenu à l'aide d'un outil de synthèse de haut niveau : Vivado HLS. De plus, vous allez aussi devoir concevoir d'autres circuits comme des synchroniseurs et des détecteurs de front montant, pour gérer les interfaces physiques présentes sur le FPGA.

Pour cela, la répartition des séances de TP est la suivante :

- Séance 1 : Mise en place des circuits de gestion des interfaces physiques
 - Conception et validation d'un synchroniseur
 - Conception et validation d'un détecteur de front montant.
 - Intégration et validation d'un circuit basé sur le synchroniseur, le détecteur de front montants et d'un contrôleur d'afficheur 4 x 7-segments.
- Séance 2 : Implémentation d'un algorithme du PGCD à l'aide de la méthodologie ASMD
 - Utilisation de la méthode de conception ASMD pour implémenter l'algorithme du PGCD, sous la forme d'une *Finite State Machine with Datapath* (FSMD).
 - Développement et validation du circuit numérique correspondant à la FSMD obtenue.
- Séance 3 : Implémentation d'un algorithme du PGCD à l'aide de la synthèse de haut niveau
 - Conception et validation d'un circuit numérique implémentant l'algorithme du PGCD à l'aide de Vivado HLS.
- Séance 4 : Intégration des FSMDs dans un circuit
 - Intégration et validation d'un circuit intégrant les interfaces physiques, la FSMD correspondant au PGCD et d'un contrôleur d'afficheur 4x7-segments.

Algorithme du PGCD

L'algorithme du PGCD permet d'obtenir le plus grand entier positif **c=PGCD(a, b)** permettant de diviser deux nombres entiers positifs : **a** et **b**. Par exemple, le PGCD de 12 et 15 est 3, car il n'existe pas de nombres plus grands que 3 permettant de diviser 12 et 15 et d'obtenir un résultat entier.

L'algorithme du PGCD peut s'exprimer récursivement de la manière suivante :

$$PGCD(a, b) = \begin{cases} PGCD(a - b, b) & \text{si } a > b \\ PGCD(a, b - a) & \text{si } a < b \\ a & \text{si } a = b \end{cases}$$

Présentation de la plateforme utilisée

La Basys 3, produit par Digilent, est une carte de développement FPGA basée sur un Artix-7 (Xilinx/AMD). Elle dispose de plusieurs interfaces physiques permettant à l'utilisateur d'interagir avec le circuit implémenté sur le FPGA :

- Sorties :
 - 1 afficheur 4 x 7-segments
 - 16 LEDs utilisateurs
 - 1 connecteur VGA
- Entrées :
 - 16 commutateurs (*switchs*)
 - 5 boutons utilisateurs
- Entrées/sorties :
 - 3 connecteurs PMOD
 - 1 connecteur USB

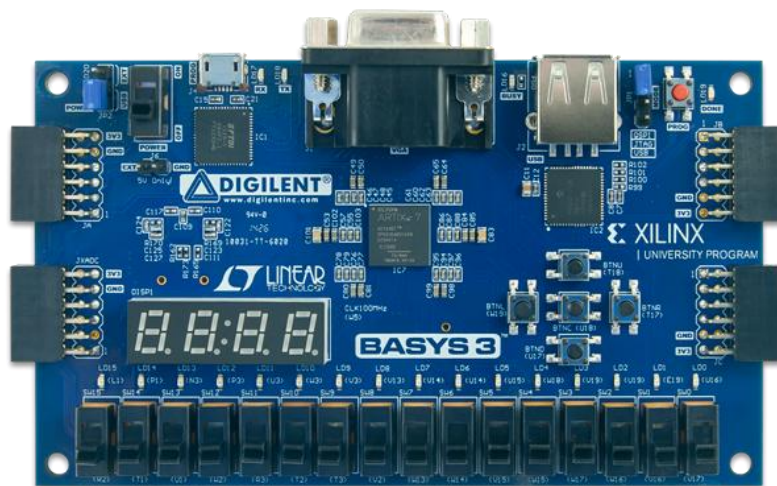


Photo vue du dessus de la carte de développement Basys 3 (Digilent.com)

Le circuit que vous allez implémenter dans ces séances de TP utilisent les interfaces physiques suivantes :

- 8 commutateurs (SW0-SW7), pour indiquer le 1^{er} nombre nécessaire au calcul du PGCD.
- 8 commutateurs (SW8-SW15), pour indiquer le 2^{ème} nombre nécessaire au calcul du PGCD.
- 1 bouton utilisateur (BTNC), pour indiquer le démarrage du calcul du PGCD (*start*).
- 1 bouton utilisateur (BTNR), pour la réinitialisation des circuits synchrones (*reset*).
- 1 afficheur 4 x 7-segments, pour afficher le résultat du PGCD.

Séance 1 : Mise en place des circuits de gestion des interfaces physiques

Cette première séance de travaux pratiques est dédiée à la gestion des interfaces physiques (boutons, commutateurs et afficheur 4 x 7-segments) utilisés par l'utilisateur pour interagir avec le circuit permettant de calculer le PGCD.

1.1 Conception et validation d'un synchroniseur en VHDL

Pour cela, il faut d'abord concevoir un synchroniseur (**synchronizer.vhd**), qui sera relié au bouton *start* (BTNC), évitant de propager des métastabilités au reste des circuits synchrones. En effet, le bouton *start* étant contrôlé par l'utilisateur (n'ayant pas de notion de l'horloge interne), il risque de propager des métastabilités dans le reste du circuit.

- 1.1.1 Rappeler le principe de fonctionnement d'un synchroniseur.
- 1.1.2 Concevoir le code VHDL d'un synchroniseur, avec un *reset* asynchrone.
- 1.1.3 Concevoir un *testbench* VHDL **synchronizer_tb.vhd** pour valider le bon fonctionnement du synchroniseur.

1.2 Conception d'un détecteur de front montant à l'aide d'une machine de Moore

Pour détecter l'appui sur le bouton *start* (BTNC), à la sortie du synchroniseur, il faut utiliser un détecteur de front montant. Dans un premier temps, vous allez concevoir un détecteur de front montant (**red_moore.vhd**), à l'aide d'une machine à états de Moore (sorties inconditionnelles).

- 1.2.1 Donner le diagramme d'états de la machine à états de Moore.
- 1.2.2 Concevoir le code VHDL du détecteur de front montant, avec un *reset* asynchrone.
- 1.2.3 Concevoir un *testbench* VHDL **red_tb.vhd** pour valider le bon fonctionnement du détecteur.

1.3 Conception d'un détecteur de front montant à l'aide d'une machine de Mealy

Dans un second temps, vous allez concevoir un détecteur de front montant (**red_mealy.vhd**), à l'aide d'une machine à états de Mealy (sorties conditionnelles), puis valider et comparer son fonctionnement par rapport au détecteur de front montant basé sur une machine à états de Moore.

- 1.3.1 Donner le diagramme d'états de la machine à états de Mealy.
- 1.3.2 Concevoir le code VHDL du détecteur de front montant, avec un *reset* asynchrone.
- 1.3.3 Valider et comparer le fonctionnement du détecteur à machine à états de Moore avec le détecteur à machine à états de Mealy, à l'aide du *testbench* VHDL précédent.

Question : Quelle est la différence entre le détecteur de front montant basé sur une machine à états de Moore et celui basé sur une machine à états de Mealy ?

1.4 Intégration et test des interfaces physiques

Cette dernière partie consiste à créer un circuit (**top_physical.vhd**), intégrant le synchroniseur, le détecteur de front montant (de Moore ou de Mealy, à vous de choisir) et le contrôleur d'afficheur 4 x 7-segments (décrit par le code **SevenSegmentsPeripheral.vhd**, disponible sur Chamilo). Ce circuit affichera sur l'afficheur 4 x 7-segments les 2 entrées de 8 bits, respectivement *input1* (SW0-SW7) et *input2* (SW8-SW15), en fonction de l'entrée *start* permettant de mettre à jour l'affichage.

- 1.4.2 Concevoir le code VHDL du circuit décrit.
- 1.4.3 Concevoir un *testbench* VHDL **top_tb.vhd** pour valider le bon fonctionnement du circuit décrit.
- 1.4.4 Intégrer et valider le bon fonctionnement du circuit sur la carte de développement Basys 3.

Séance 2 : Implémentation d'un algorithme du PGCD à l'aide de la méthodologie ASMD (1/2)

Cette seconde séance de travaux pratiques est dédiée à l'implémentation de l'algorithme du PGCD à l'aide de la méthodologie ASMD, vue en cours. Plus particulièrement, vous comparerez deux FSMs à l'aide de cette même méthode. D'une part, une FSM à sorties inconditionnelles (Moore). D'autre part, une FSM à sorties conditionnelles (Mealy). Enfin, vous comparerez le fonctionnement des deux FSMs.

Remarque : Si vous n'avez pas fini le travail demandé lors de la séance précédente, vous pouvez recommencer là où vous en étiez lors de la séance précédente.

2.0 Préparation à faire avant la séance de TP

- 2.0.1 A partir de la version récursive de l'algorithme du PGCD, introduite au début du document, donner la version itérative de l'algorithme du PGCD.
- 2.0.2 A partir de la version itérative de l'algorithme du PGCD, donner l'algorithme (*flowchart*) correspondant (sans, puis avec une structure de contrôle, i.e. l'interface).
- 2.0.3 En utilisant l'algorithme obtenu et à l'aide de la méthodologie ASMD, donner la FSM correspondante, avec des sorties inconditionnelles (Moore).
- 2.0.4 En utilisant l'algorithme obtenu et à l'aide de la méthodologie ASMD, donner la FSM correspondante, avec des sorties conditionnelles (Mealy).

2.1 Obtention de la FSM, avec des sorties inconditionnelles (Moore)

La première partie de cette seconde séance de travaux pratiques consistera à développer et valider la FSM implémentant l'algorithme du PGCD (**pgcd_moore.vhd**), avec des sorties inconditionnelles (Moore).

- 2.1.1 Concevoir le code VHDL de la FSM à sorties inconditionnelles (Moore), avec un *reset* asynchrone.
- 2.1.2 Concevoir un *testbench* VHDL **pgcd_tb.vhd** pour valider le bon fonctionnement de la FSM.

2.2 Obtention de la FSM, avec des sorties conditionnelles (Mealy)

La seconde partie de cette seconde séance de travaux pratiques consistera à développer et valider la FSM implémentant l'algorithme du PGCD (**pgcd_mealy.vhd**), avec des sorties conditionnelles (Mealy). Dans un second temps, vous comparerez le fonctionnement de la FSM à sorties inconditionnelles (Moore) au fonctionnement de la FSM à sorties conditionnelles (Mealy).

- 2.2.1 Concevoir le code VHDL de la FSM à sorties conditionnelles (Mealy), avec un *reset* asynchrone.
- 2.2.2 Valider et comparer le fonctionnement de la FSM à sorties inconditionnelles (Moore) avec la FSM à sorties conditionnelles (Mealy), à l'aide du *testbench* VHDL précédent.

Question : Quelle est la différence entre la FSM à sorties inconditionnelles (Moore) et la FSM à sorties conditionnelles (Mealy) ?

Séance 3 : Implémentation d'un algorithme du PGCD à l'aide de la synthèse de haut niveau

Cette troisième séance est dédiée à l'implémentation de l'algorithme du PGCD à l'aide d'un logiciel de synthèse de haut niveau : Vivado HLS. Dans un premier temps, vous allez utiliser Vivado HLS pour obtenir une implémentation VHDL de l'algorithme du PGCD. Enfin, vous allez comparer le fonctionnement de l'implémentation obtenue, avec les implémentations de l'algorithme du PGCD que vous avez obtenues précédemment.

Remarque : Si vous n'avez pas fini le travail demandé lors de la séance précédente, vous pouvez recommencer là où vous en étiez lors de la séance précédente.

3.0 Préparation à faire avant la séance de TP

- 3.0.1 Regarder la [vidéo](#), présentant le fonctionnement de l'outil de synthèse de haut niveau (HLS) : Vitis HLS 2020.2.
- 3.0.2 Donner le code C de la fonction `uint32_t pgcd_hls(uint32_t, uint32_t)`, correspondant à la version itérative de l'algorithme du PGCD.

Remarque : La vidéo qui vous est demandée de regarder présente le fonctionnement de Vitis HLS 2020.2, alors que le logiciel de synthèse de haut niveau que vous allez utiliser est Vivado HLS 2019.2. Cependant, les deux logiciels et leurs interfaces utilisateurs sont très similaires, ils partagent notamment les mêmes étapes (simulation C, synthèse, co-simulation et export RTL).

3.1 Configuration du projet Vivado HLS

La première partie de cette séance consiste à créer et configurer un projet Vivado HLS.

- 3.1.1 Créer un projet Vivado HLS, intitulé **pgcd_hls**, en spécifiant uniquement la carte de développement utilisée (Basys 3) et la fréquence de fonctionnement (100 MHz, correspondant à une période de 10 ns).
- 3.1.2 Dans le menu principal, dans l'onglet *Explorer*, ajouter un fichier **pgcd_hls.c** (clique-droit sur *Sources*, puis *New Files ...*). Une fois le fichier créé, ajouter le code C obtenu lors de la préparation.
- 3.1.3 Dans le menu principal, dans l'onglet *Explorer*, ajouter un fichier **pgcd_hls_tb.c** (clique-droit sur *Testbench*, puis *New Files ...*). Une fois le fichier créé, écrivez un *testbench* simple, permettant d'afficher le PGCD de 12 et de 15.
- 3.1.4 Dans la barre d'onglets, cliquer sur l'onglet *Settings*, puis *Synthesis*, indiquer que la fonction à synthétiser est la fonction **pgcd_hls** du fichier **pgcd_hls.c**.

3.2 Conception d'une implémentation matérielle du PGCD à l'aide de Vivado HLS

La seconde partie de cette séance consiste à générer le code VHDL correspondant au code C obtenu lors de la préparation de votre TP, à l'aide de Vivado HLS.

- 3.2.1 Dans la barre d'onglets, cliquer sur l'onglet *C simulation* et lancer la simulation C permettant de valider le bon fonctionnement de votre fonction
- 3.2.2 Dans la barre d'onglets, cliquer sur *Synthesis* pour lancer la synthèse de haut niveau. Vous pourrez inspecter le code VHDL généré dans l'onglet *Explorer* :
`solution1/syn/vhdl/pgcd_hls.vhd`
- 3.2.3 Dans la barre d'onglets, cliquer sur *Co-simulation* pour lancer la co-simulation C/RTL.

Question : A l'aide de la vue *Analyzer* de Vivado HLS, analyser l'ordonnancement de votre code C et comparer le à l'ordonnancement des ASMDs de la séance précédente. Que remarquez-vous ?

3.3 Validation du code VHDL obtenu à l'aide de la synthèse de haut niveau

La troisième partie de cette séance consiste à valider et à comparer le fonctionnement du code VHDL obtenu à l'aide de Vivado HLS (**pgcd_hls.vhd**) et des FSMs obtenues lors de la séance précédente.

3.3.1 Valider et comparer le fonctionnement du code VHDL obtenu avec les FSMs obtenues précédemment.

Question : Quelle est la différence de fonctionnement entre le circuit obtenu à l'aide de Vivado HLS et les FSMs obtenues lors de la séance précédente ? Est-ce que c'était prévisible ?

3.4 Pour aller plus loin

Dans cette dernière partie, vous allez implémenter une autre version de l'algorithme du PGCD, basée sur une opération de modulo à la place d'une opération de soustraction. Vous allez donc utiliser Vivado HLS pour synthétiser le code C ci-dessous (**pgcd_modulo_hls.c**) et obtenir le code VHDL **pgcd_modulo_hls.vhd**.

Pour cela, vous allez suivre les étapes décrites précédemment. Plus particulièrement, vous allez créer un projet **pgcd_modulo_hls**, ajouter le code **pgcd_modulo_hls.c** et créer un *testbench* **pgcd_modulo_hls_tb.c** (basée sur **pgcd_hls_tb.c**). Ensuite, réaliser la simulation C, puis la synthèse et enfin la co-simulation C/RTL.

Question : Quelle est la différence entre **pgcd_hls.vhd** et **pgcd_modulo_hls.vhd** en termes d'utilisation du nombre de ressources (LUT et FF) et de latence ? A votre avis, à quoi est dû cette différence ? Que concluez-vous sur la forme du code C fourni en entrée de Vivado HLS ?

```
#include <stdint.h>

uint32_t pgcd_modulo_hls(uint32_t a, uint32_t b){
    uint32_t t ;
    while (b != 0){
        t=b;
        b=a%b;
        a=t;
    }
    return b;
}
```

Séance 4 : Intégration des FSMs dans un circuit

Cette dernière séance de travaux pratiques est dédiée aux tests, sur carte de développement Basys 3, des FSMs obtenues lors de la deuxième et troisième séance, à l'aide des composants (synchroniseur, détecteur de front montant, contrôleur 4 x 7-segments) obtenus lors de la première séance. Pour cela, vous allez créer un circuit intégrant ces différents composants, comme décrit dans l'introduction du document.

Remarque : Si vous n'avez pas fini le travail demandé lors de la séance précédente, vous pouvez recommencer là où vous en étiez lors de la séance précédente.

4.1 Développement d'un circuit basé sur une FSM à sorties inconditionnelles (Moore)

La première partie consiste à créer un circuit (**top_moore.vhd**), intégrant le synchroniseur, le détecteur de front montant (à base d'une machine à états de Moore), le contrôleur d'afficheur 4 x 7 segments et la FSM à sorties inconditionnelles (Moore). Ce circuit affichera sur l'afficheur 4 x 7-segments le résultat du calcul du PGCD sur les 2 entrées de 8 bits, respectivement *input1* (SW0-SW7) et *input2* (SW8-SW15), en fonction de l'entrée *start* permettant de démarrer le calcul du PGCD.

- 4.1.1 Concevoir le code VHDL du circuit décrit.
- 4.1.2 Valider le fonctionnement du circuit à l'aide du testbench VHDL **top_tb.vhd**.
- 4.1.3 Intégrer et valider le bon fonctionnement du circuit sur la carte de développement Basys 3.

4.2 Développement d'un circuit basé sur une FSM à sorties conditionnelles (Mealy)

La seconde partie consiste à créer un circuit (**top_mealy.vhd**), intégrant le synchroniseur, le détecteur de front montant (à base d'une machine à états de Mealy), le contrôleur d'afficheur 4 x 7 segments et la FSM à sorties conditionnelles (Mealy). Ce circuit affichera sur l'afficheur 4 x 7-segments le résultat du calcul du PGCD sur les 2 entrées de 8 bits, respectivement *input1* (SW0-SW7) et *input2* (SW8-SW15), en fonction de l'entrée *start* permettant de démarrer le calcul du PGCD.

- 4.2.1 Concevoir le code VHDL du circuit décrit
- 4.2.2 Valider le fonctionnement du circuit à l'aide du testbench VHDL **top_tb.vhd**.
- 4.2.3 Intégrer et valider le bon fonctionnement du circuit sur la carte de développement Basys 3.

4.3 Développement d'un circuit basé sur le code VHDL obtenu à l'aide de la synthèse de haut niveau

La troisième partie consiste à créer un circuit intégrant le synchroniseur, le détecteur de front montant (à base d'une machine de Mealy), le contrôleur d'afficheur 4 x 7 segments et le code VHDL obtenu à l'aide de Vivado HLS. Ce circuit affichera sur l'afficheur 4 x 7-segments le résultat du calcul du PGCD sur les 2 entrées de 8 bits, respectivement *input1* (SW0-SW7) et *input2* (SW8-SW15), en fonction de l'entrée *start* permettant de mettre à jour l'affichage.

- 4.3.1 Concevoir le code VHDL du circuit décrit.
- 4.3.2 Valider le fonctionnement du circuit à l'aide du testbench VHDL **top_tb.vhd**.
- 4.3.3 Intégrer et valider le bon fonctionnement du circuit sur la carte de développement Basys 3.

Question : Quelles sont les différences, en termes de ressources (LUT, FF, ...) et de performances (fréquences d'horloge maximum, ...), entre les différents circuits réalisés ?

Compte-rendu

Pour cette série de TP, un seul compte-rendu vous sera demandé. Plus particulièrement, il devra comporter entre 10 et 15 pages (maximums) et il devra contenir le contenu de votre travail préparatoire, la réponse aux différentes questions qui vous ont été posées, ainsi que des résultats que vous jugerez pertinents (simulations, ...). Concernant la date de rendu, un seul compte-rendu par monôme/binôme/trinôme devra être déposé sur Chamilo, un jour après la date de la dernière séance de TP (des deux groupes). Enfin, comme indiqué lors de la séance d'introduction, une partie de la note prendra en compte votre participation/comportement en séance.